

El Puente Frontend-Backend: Arquitectura de Validación Profesional

Los formularios son el punto de contacto crítico donde la intención del usuario se encuentra con la base de datos.



El Problema: Depender solo del backend genera una experiencia lenta y frustrante. El usuario no debería esperar la respuesta del servidor para saber que olvidó un '@' en su email.

Frontend (El Filtro): Experiencia de usuario (UX). Validación inmediata sin latencia de red.

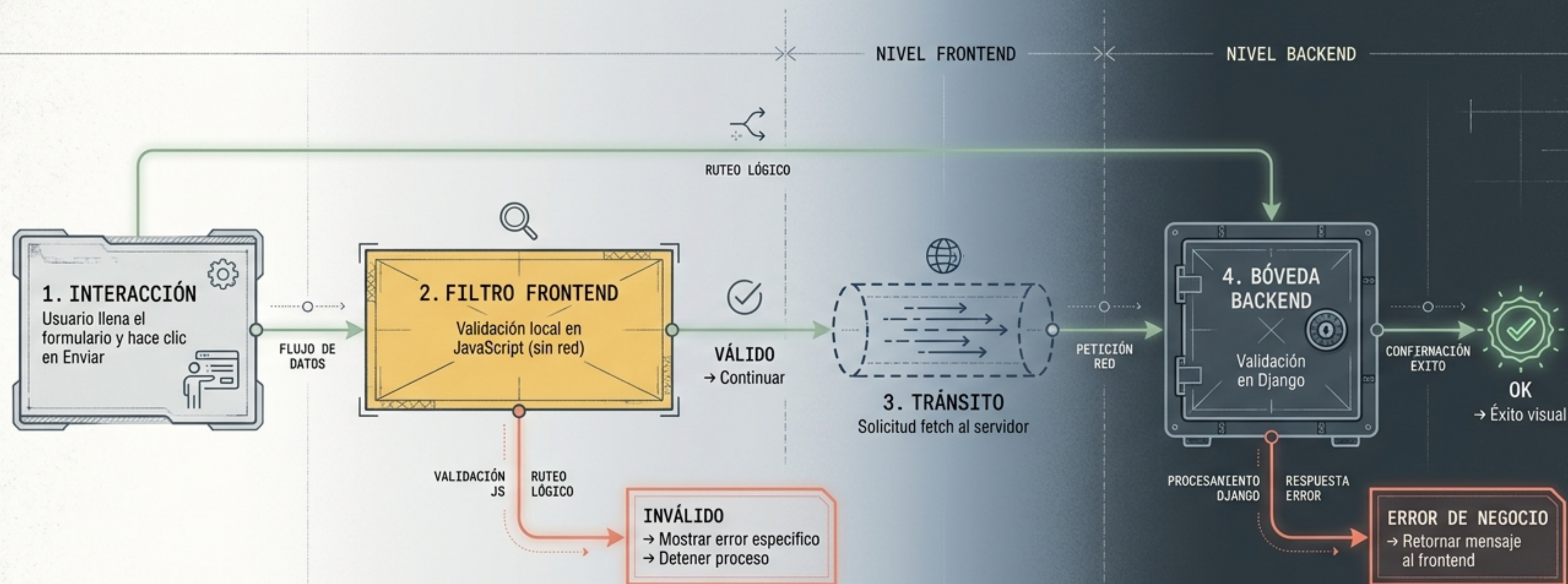
Backend (La Bóveda): Seguridad real e inquebrantable. El serializer rechaza datos inválidos categóricamente.

EL ECOSISTEMA DE VALIDACIÓN DE DATOS

PLANTA DE PROCESO

ESCALA 1:1

FECHA DE REVISIÓN



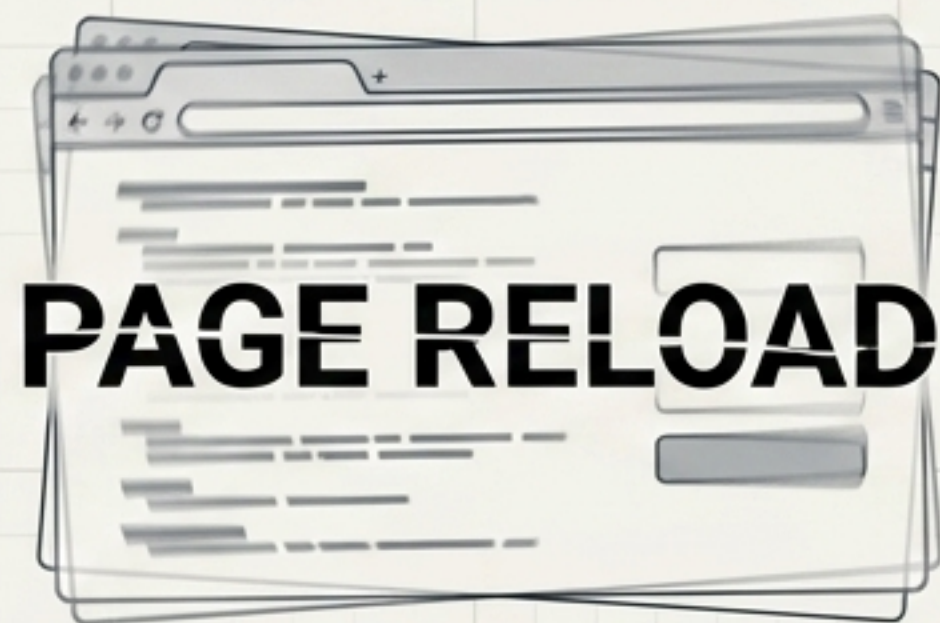
PLANTA DE PROCESO

ESCALA 1:1

FECHA DE REVISIÓN

Intercepción de la Navegación: preventDefault

El comportamiento HTML por defecto al hacer submit es recargar la página. En una arquitectura moderna, esto es destructivo: borra los datos, el estado local y la sesión visible.



```
const form = document.querySelector('#form-login')

form.addEventListener('submit', (event) => {
  event.preventDefault() // Detiene la recarga nativa
  // El control pasa a JavaScript (fetch, validación)
  console.log('Formulario enviado sin recargar')
})
```


Estrategias de Captura: Leyendo el DOM

Estrategia	Método JS	Escalabilidad	Cuándo usar
Individual	<code>document.querySelector('#id').value</code>	Baja (Repetitivo)	Formularios de 1-2 campos (ej. un buscador).
Masiva	<code>new FormData(form)</code>	Alta (Dinámica)	Formularios grandes. Extrae datos usando el atributo name del HTML.
Estructurada	<code>Object.fromEntries(...)</code>	Máxima (Lista para API)	Para enviar datos en formato JSON moderno. Convierte FormData en un objeto plano.

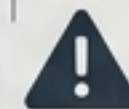
Modernizando la Extracción de Datos

```
<input type='text' id='username' name='username' />
```

```
const formData = new FormData(form)
const datos = Object.fromEntries(formData)
```

```
{ 'username': 'jhon', 'password': '1234' }
```

Callout

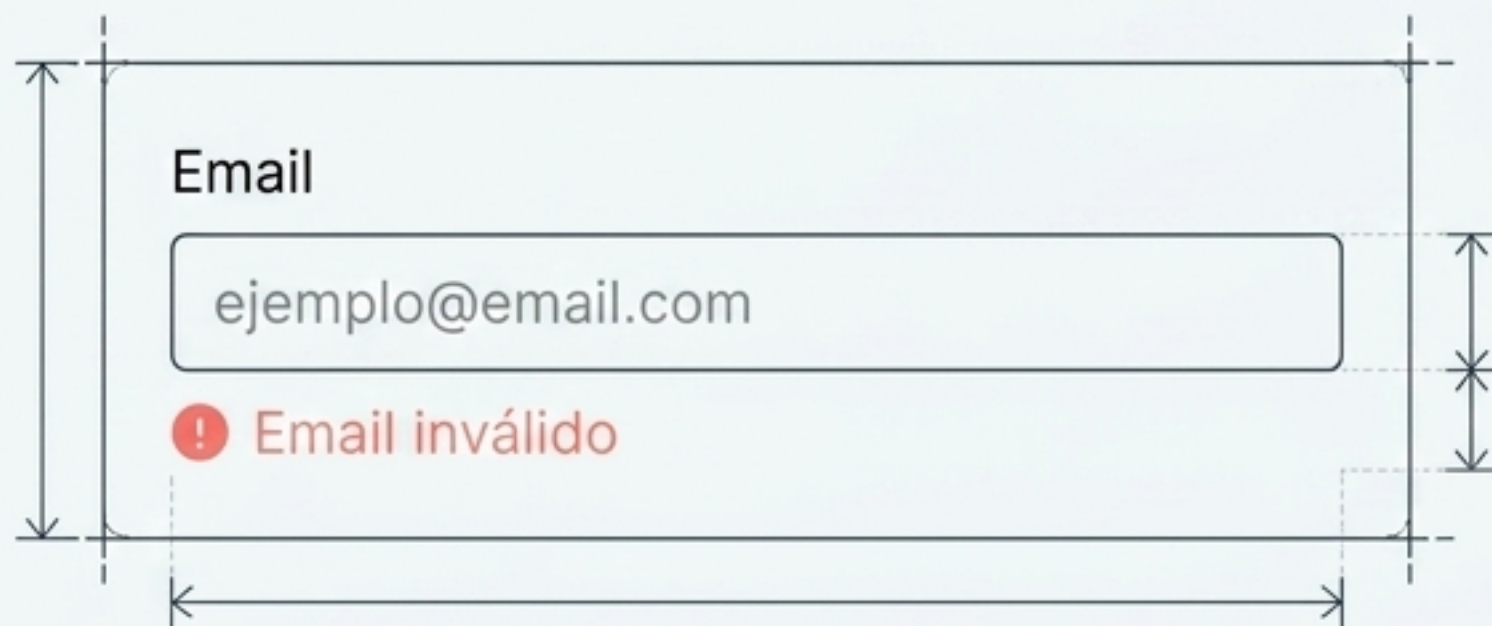


Nota Crítica: FormData es completamente ciego si los inputs no tienen el atributo name. El id no es suficiente para la recolección masiva.

Arquitectura HTML para Feedback Específico

La regla de oro de la UX: El error debe aparecer justo debajo del campo que falló, no en una alerta global.

Para esto, necesitamos anclas en el HTML.



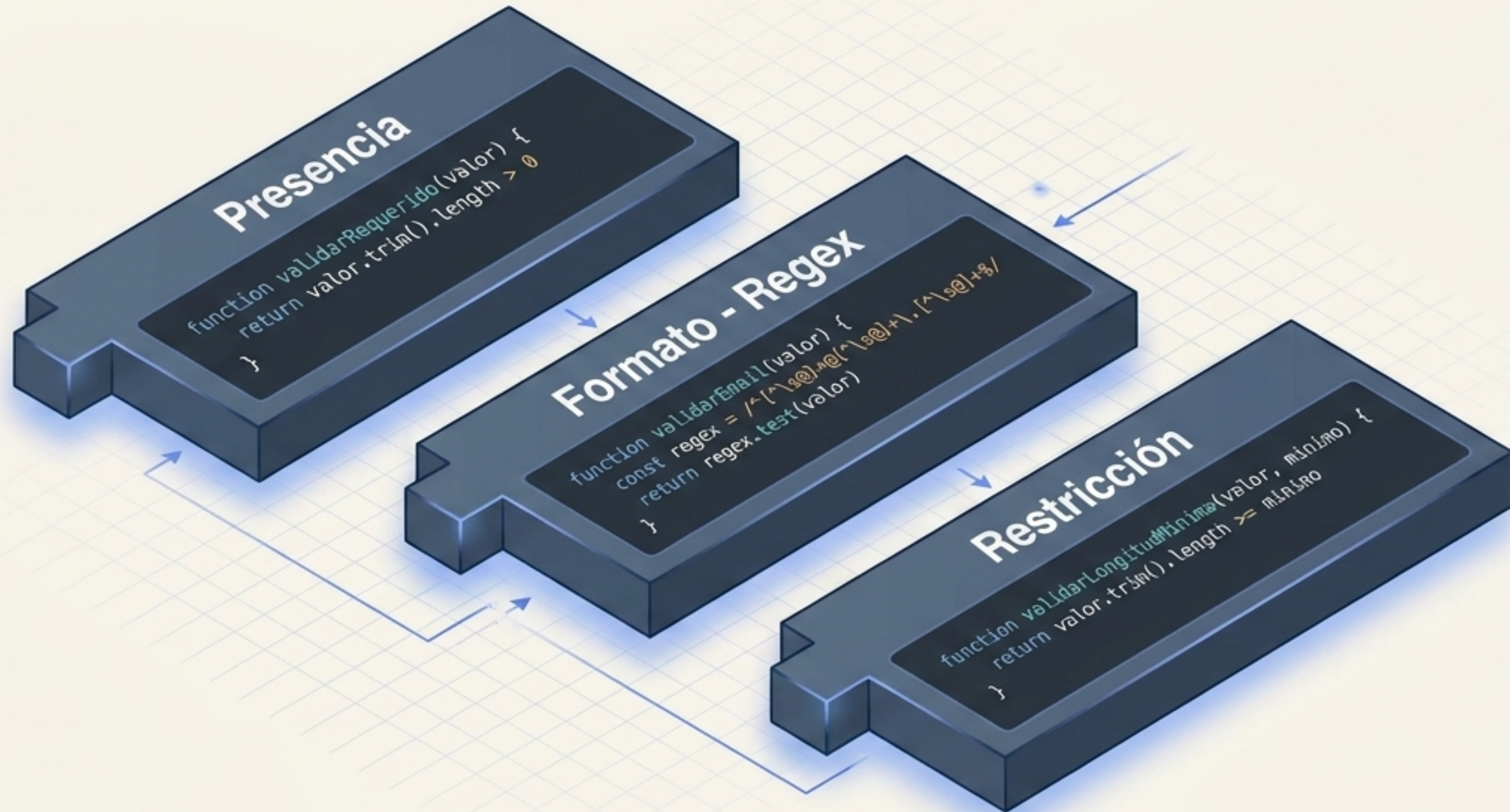
```
<div class='campo'>  
  <label for='email'>Email</label>  
  <input type='email' id='email' name='email' />
```

```
<!-- El contenedor clave y vacío por defecto: -->  
  <span class='error-msg' id='error-email'></span>  
</div>
```

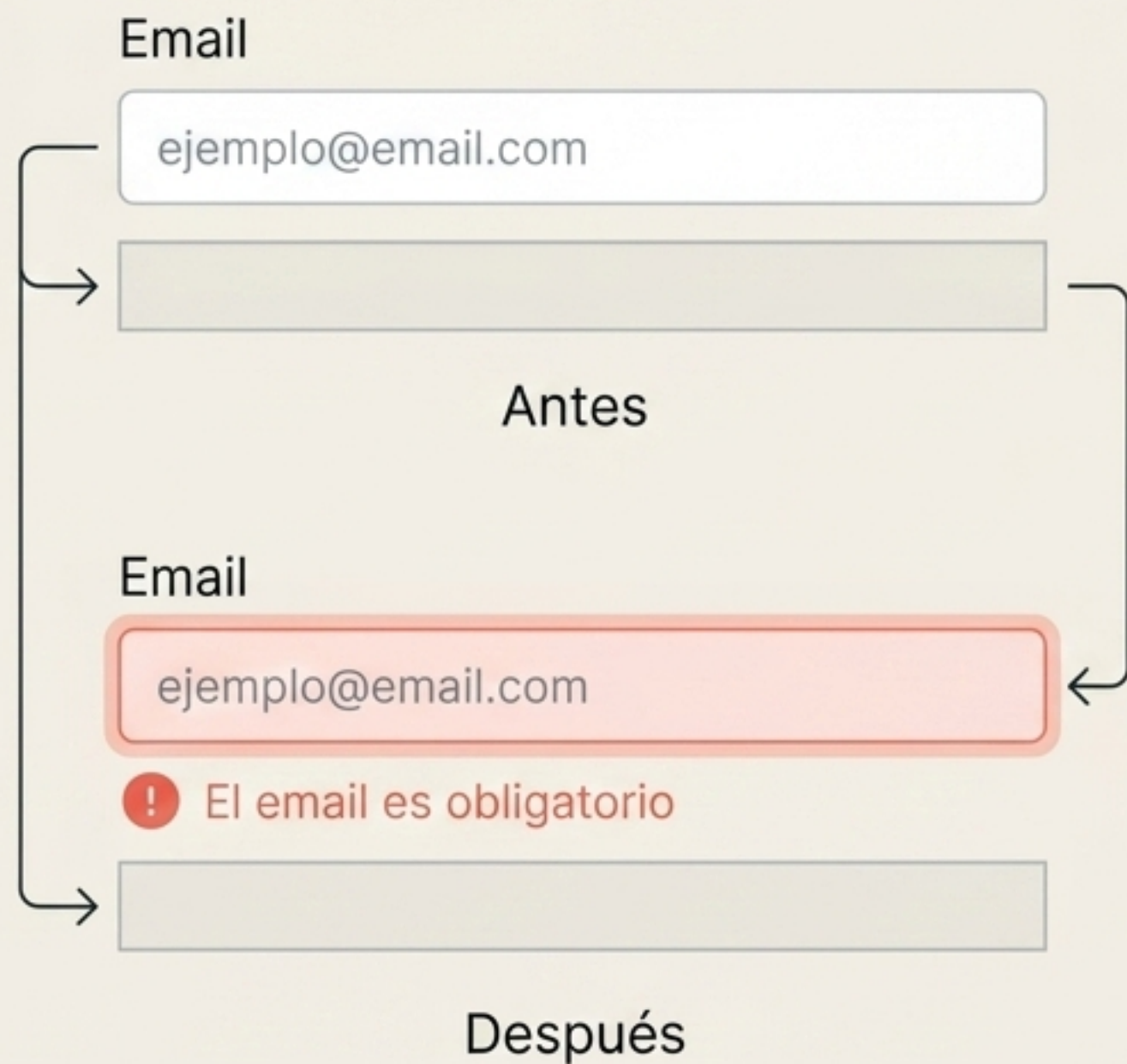
Permanece invisible hasta que JavaScript inyecta texto y añade una clase CSS.

Funciones de Validación Modulares

Dividir y vencer. Cada regla de negocio se abstrae en una función independiente que retorna un booleano (true/false).



Manipulación del DOM: Inyectando Feedback



```
// Mostrar Error
function mostrarErrorCampo(idError, mensaje) {
  const errorEl = document.querySelector(`#${idError}`);
  errorEl.textContent = mensaje;
  errorEl.classList.c = mensaje;
  errorEl.classList.add('visible');
}

// Limpiar Error
function limpiarErrorCampo(idError) {
  const errorEl = document.querySelector(`#${idError}`);
  errorEl.textContent = '';
  errorEl.classList.remove('visible');
}
```

Contexto: limpiarErrores() hace un barrido general de .error-msg antes de cada nuevo intento.

El Validador Maestro y el Diccionario de Errores

La validación revisa todo el formulario a la vez antes de detener el flujo, acumulando fallas en un objeto de configuración.



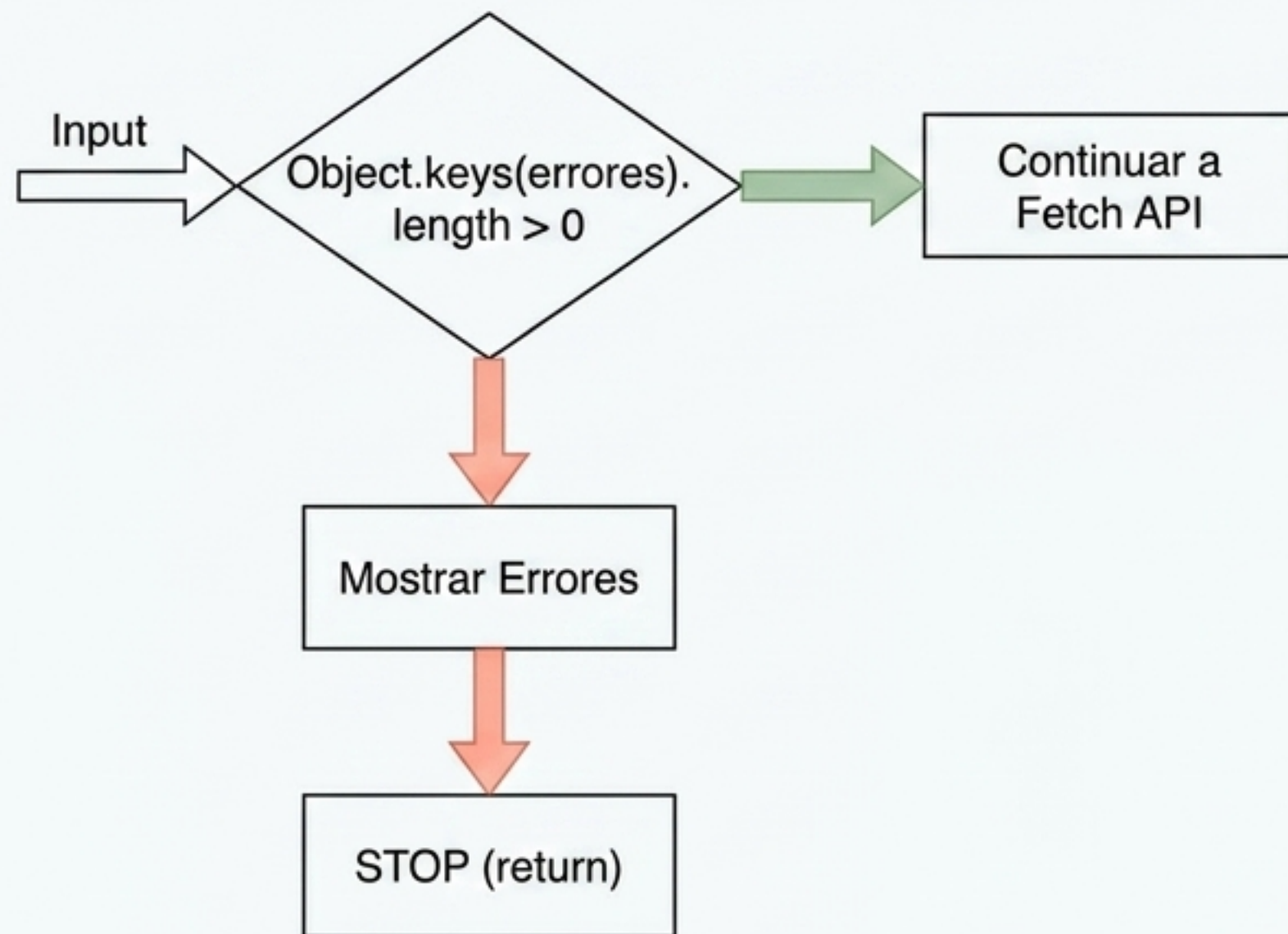
```
function validarFormularioRegistro(datos) {
  const errores = {} // Contenedor de fallas

  if (!validarRequerido(datos.username)) {
    errores.username = 'El nombre de usuario es obligatorio'
  } else if (!validarLongitudMinima(datos.username, 3)) {
    errores.username = 'Mínimo 3 caracteres'
  }

  if (datos.password !== datos.confirmar_password) {
    errores.confirmar_password = 'Las contraseñas no coinciden'
  }

  return errores
}
```


Ensamblando el Flujo: El Evento Submit



```
form.addEventListener('submit', async (event) => {
  event.preventDefault()
  limpiarErrores()

  const datos = Object.fromEntries(new FormData(form))
  const errores = validarFormularioRegistro(datos)

  // Evaluación del diccionario:
  if (Object.keys(errores).length > 0) {
    Object.entries(errores).forEach([campo, mensaje]) => {
      mostrarErrorCampo(`error-${campo}`, mensaje)
    })
    return // Interrupción temprana
  }

  // Sin errores locales – se avanza a la API
})
```


Anatomía del Error DRF (Backend)

Incluso con validación frontend, el backend tiene la última palabra. Django REST Framework estandariza los errores retornando un objeto JSON mapeado por campos.

```
HTTP/1.1 400 Bad Request
Content-Type: application/json
```

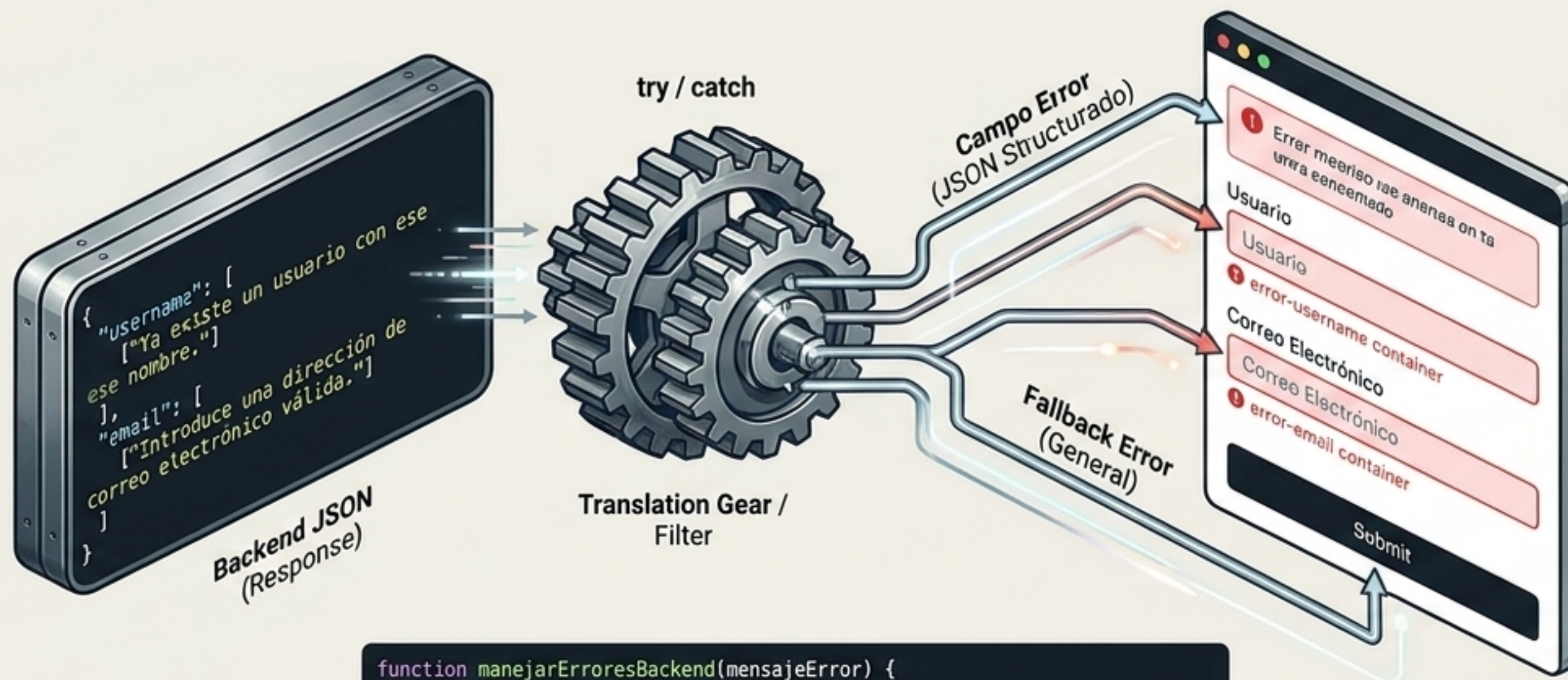
```
{
  'username': [
    'Ya existe un usuario con ese nombre.'
  ],
  'email': [
    'Introduce una dirección de correo electrónico válida.'
  ]
}
```

Nota Arquitectónica:

Los errores llegan en Arrays porque un mismo campo puede tener múltiples violaciones simultáneas en el backend.

Maapeo Inverso: Traduciendo el Backend a la Interfaz

Procesamos las excepciones del fetch. Inyectamos los errores de red en los mismos contenedores HTML locales.



```
function manejarErroresBackend(mensajeError) {  
  try {  
    const errores = JSON.parse(mensajeError)  
    Object.entries(errores).forEach([campo, mensajes] => {  
      const mensaje = Array.isArray(mensajes) ? mensajes[0] : mensajes  
      mostrarErrorCampo(`error-${campo}`, mensaje)  
    })  
  } catch {  
    mostrarErrorGeneral(mensajeError)  
  }  
}
```

Arquitectura de Traducción:
Convertimos la respuesta de error del backend en actualizaciones de interfaz de usuario específicas.

Feedback Visual y Gestión de Ansiedad (UX)

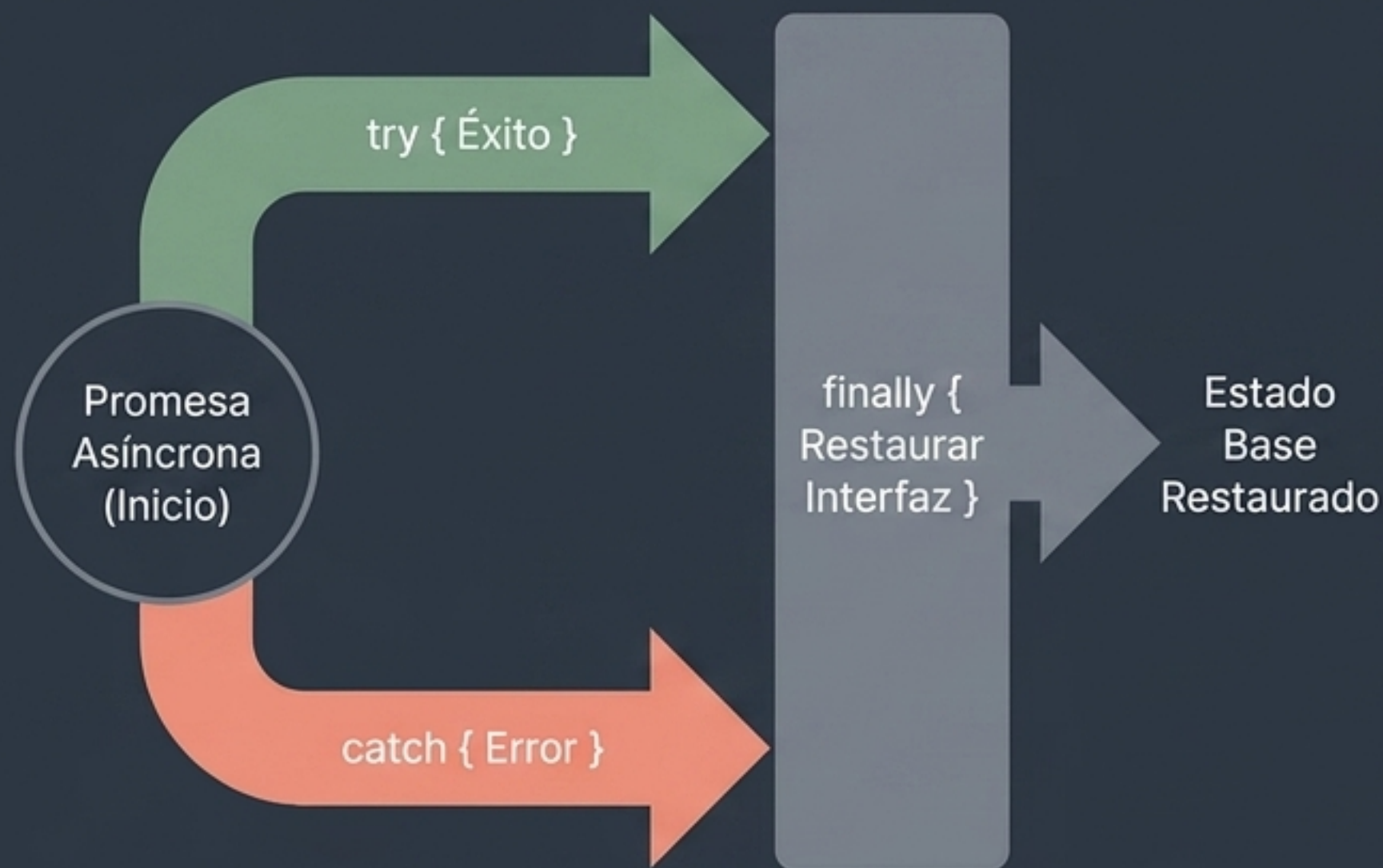
Un formulario asíncrono debe comunicar claramente su **estado** para evitar **envíos múltiples** (doble-click de pánico).



```
function setEstadoCargando(cargando) {  
  if (cargando) {  
    btnSubmit.disabled = true // Previene envíos dobles  
    btnSubmit.textContent = 'Enviando...'  
    btnSubmit.classList.add('loading')  
  } else {  
    btnSubmit.disabled = false  
    btnSubmit.textContent = 'Registrarse'  
    btnSubmit.classList.remove('loading')  
  }  
}
```


La Importancia Crítica del Bloque Finally

Garantizar la restauración del estado base sin importar el resultado de la red.



```
setEstadoCargando(true) // 1. Bloquear interfaz
```

```
try {  
  await registrarUsuario(datos)  
  mostrarExito('¡Registro exitoso!')  
  form.reset() // 2a. Limpiar en éxito  
} catch (error) {  
  manejarErroresBackend(error.message) // 2b. Manejar fallas  
} finally {  
  setEstadoCargando(false) // 3. Restaurar siempre  
}
```

El bloque finally asegura que el estado de carga se desactive y la interfaz vuelva a su estado normal, independientemente de si la promesa se resuelve con éxito o con un error.

Validación en Tiempo Real: La Fricción Adecuada

Validar en cada pulsación de tecla estresa al usuario. La mejor estrategia combina dos eventos distintos.

 **blur** - Pierde el foco



Momento ideal
para validar.

 **input** - Escribiendo



Momento para ser
indulgente y limpiar.

```
1 // Validar al salir
2 inputEmail.addEventListener('blur', (e) => {
3   validarEmailUnico(e.target.value)
4 })
5
6 // Limpiar al corregir
7 inputEmail.addEventListener('input', () => {
8   limpiarErrorCampo('error-email')
9 })
```